

# SHOPLY

## Flutter E-Commerce Product Catalog

### ASSIGNMENT 6

A responsive, dynamic and professional shopping experience built with Flutter.

[Github Link](#)

## Project Overview

Shoply is a Flutter-based e-commerce product catalog application designed to demonstrate responsive UI development, reusable widgets, local product data modeling, interactive catalog filtering, wishlist and cart management, and a polished shopping interface. The application presents a collection of products across Electronics, Fashion, Books and Accessories, with a warm terracotta and neutral visual identity that is intentionally different from earlier dashboard-style assignments.

### Core Objectives

- Build a modern e-commerce interface rather than a basic static product list.
- Use reusable Flutter widgets and separate model, data, screen, utility and widget layers.
- Support responsive layouts for desktop, tablet and mobile widths.
- Provide meaningful interactions: search, category filtering, sorting, wishlist and cart actions.
- Create a realistic shopping flow with product previews, cart review and checkout interaction.

### Technology Stack

|                    |                                                                |
|--------------------|----------------------------------------------------------------|
|                    |                                                                |
| Flutter + Dart     | Application UI, stateful interactions and responsive layouts   |
| Material Design    | Buttons, dialogs, icons, text fields and navigation components |
| Custom Widgets     | Hero carousel, product cards and reusable footer               |
| Local Product Data | Static ProductData list used as the catalog source             |
| Image.network      | Product imagery loaded from remote image URLs                  |

---

## 2. System Architecture

---

The application follows a lightweight layered structure. The entry point initializes the Shoply application and applies the shared theme. The catalog screen owns the current cart, wishlist, search query, category and sorting state. Product information is represented by a dedicated model and supplied through a separate data class.

|         |                             |                                                        |
|---------|-----------------------------|--------------------------------------------------------|
| Entry   | main.dart                   | Starts ShoplyApp and opens CatalogScreen               |
| Model   | models/product_model.dart   | Defines the Product data structure                     |
| Data    | data/product_data.dart      | Stores the catalog's product records                   |
| Theme   | utils/app_theme.dart        | Centralizes colors, typography and Material styles     |
| Screens | screens/catalog_screen.dart | Catalog, search, filters, wishlist and navigation      |
| Screens | screens/cart_screen.dart    | Cart display, totals, removal and checkout interaction |
| Widgets | widgets/hero_carousel.dart  | Featured product carousel and preview                  |
| Widgets | widgets/shoply_footer.dart  | Newsletter, links, trust information and footer        |

## Product Data Model

Each product is represented by a Product object containing an identifier, display name, description, price, category, image URL, rating, review count and two flags for featured and new products. This keeps the UI independent from hard-coded product-card content.

```
Product { id, name, description, price, category, imageUrl, rating, reviewCount, isFeatured, isNew }
```

## Catalog Data

The catalog contains 11 sample products. The available categories are Electronics, Fashion, Books and Accessories. Featured and new flags are used by the hero carousel, badges and sorting behavior.

## State and Filtering Logic

- Search checks product name, category and description against the entered query. • Category filtering narrows the catalog to the selected category.
- Sorting supports Featured, Price Low-to-High, Price High-to-Low, Rating and Newest. • Wishlist state stores selected Product objects and updates immediately through setState. • Cart state prevents duplicate product entries and updates the cart counter.

---

## 3. User Interface and Major Features

The interface was designed to feel like a compact modern shopping storefront. The layout uses neutral

backgrounds, white surfaces, terracotta primary actions, rounded cards and large product imagery. The design is responsive and changes the number of catalog columns according to available width.

## 1. Responsive Header

The header presents the Shoply brand and quick access to Wishlist and Cart. The interface uses responsive spacing so the controls remain usable at smaller widths.

## 2. Featured Hero Carousel

Featured products are displayed in a PageView-based carousel. It automatically rotates every four seconds, supports previous/next navigation and includes selectable indicator dots. Hovering pauses the carousel on desktop. Selecting a slide opens a product preview sheet.

## 3. Search, Categories and Sorting

The catalog can be searched using product names, categories and descriptions. Category chips allow fast filtering, while the sort dropdown changes the ordering of visible products.

## 4. Product Cards

Each card shows the product image, NEW badge where applicable, wishlist control, product information, rating and cart action. Product states are reflected immediately in the interface.

---

# 4. Shopping Interactions and Responsive Design

### Wishlist

Users can add or remove products from the wishlist using the heart control. The wishlist is accessible from the header and presents saved products in a dedicated interactive view. Products can also be moved into the cart from the wishlist.

### Cart

The cart is implemented as a dedicated CartScreen. It displays the selected products, allows individual removal, calculates the subtotal and applies a delivery rule: delivery is free when the subtotal reaches ₹5,000; otherwise a ₹99 delivery charge is applied. The total is then presented in an order summary.

### Checkout Interaction

The cart includes a checkout action that opens a confirmation dialog. The current assignment keeps payment and delivery integration as a UI-level simulation rather than connecting to a real payment gateway or backend service.

### Responsive Layout

| ≥ 1250 px   | 4 columns |
|-------------|-----------|
| 850–1249 px | 3 columns |
| 550–849 px  | 2 columns |
| < 550 px    | 1 column  |

## Footer and Supporting UI

The reusable ShoplyFooter provides brand information, navigation links, social buttons, newsletter subscription validation, trust information and a bottom copyright area. The footer uses the same responsive principles as the main catalog.

## Code Quality and Reusability

- Product, theme, data and screens are separated into focused files.
  - Reusable UI elements reduce duplication across the application.
  - Stateful widgets are used only where interaction requires mutable UI state.
  - Error builders are provided for network images so broken images do not crash the layout.
- 

# 5. Testing, Results and Future Scope

## Testing Performed

| Application launch    | Shoply opens successfully through main.dart                      |
|-----------------------|------------------------------------------------------------------|
| Hero carousel         | Featured products rotate and navigation controls work            |
| Search                | Catalog updates according to matching product text               |
| Category filter       | Products are filtered by selected category                       |
| Sorting               | Products reorder by price, rating, featured state or newest flag |
| Wishlist              | Products can be added/removed and the counter updates            |
| Cart                  | Products can be added, removed and totals are recalculated       |
| Responsive layout     | Catalog changes from 4/3/2/1 columns by width                    |
| Network image failure | Fallback image UI is displayed instead of breaking the card      |
| Footer newsletter     | Basic email validation and subscribed state work                 |

## Result

The completed Shoply application provides a functional front-end e-commerce experience rather than a static mock-up. It demonstrates Flutter fundamentals together with practical UI engineering: componentization, state updates, responsive layout decisions, reusable widgets and user feedback.

## Learning Outcomes

- Improved understanding of StatefulWidget and setState-driven UI updates.
- Practical use of ListView, GridView, CustomScrollView, SliverGrid and PageView.
- Responsive design using MediaQuery and available layout constraints.

- Separation of data models from presentation code.
- Building reusable custom widgets and shared application styling.
- Designing realistic e-commerce interaction flows such as wishlist and cart management.

## Future Enhancements

- Connect the product catalog to a REST API or Firebase backend.
- Add quantity controls and persistent cart storage.
- Implement user authentication and saved addresses.
- Add real payment gateway integration and order history.
- Add product detail routes, reviews and related-product recommendations.
- Persist wishlist and cart data using SQLite, Hive or another local database.

## Conclusion

Shoply successfully demonstrates how Flutter can be used to create a polished, responsive and interactive e-commerce interface. The project combines a clean architecture with practical shopping interactions and provides a strong foundation for a production-style store application.